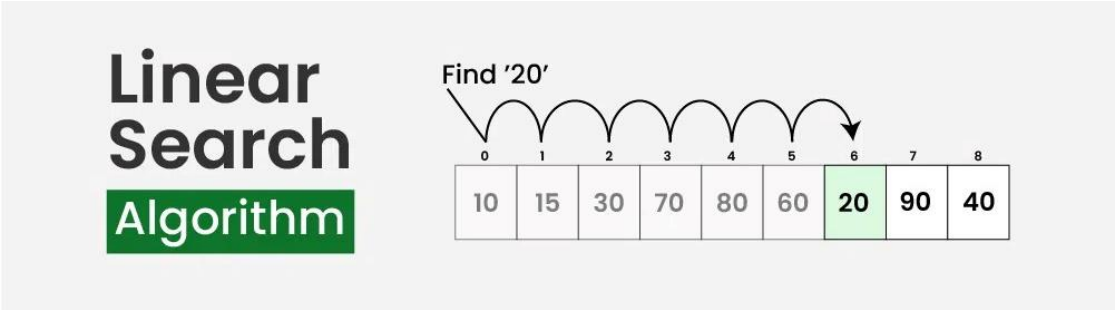
Unit V: Searching & Sorting

Linear Search is defined as a sequential search algorithm that starts at one end and goes through each element of a list until the desired element is found, otherwise the search continues till the end of the data set.



What is Linear Search?

Linear Search is a method for searching an element in a collection of elements. In Linear Search, each element of the collection is visited one by one in a sequential fashion to find the desired element. Linear Search is also known as **Sequential Search.**

Algorithm for Linear Search:

The algorithm for linear search can be broken down into the following steps:

- **Start:** Begin at the first element of the collection of elements.
- **Compare:** Compare the current element with the desired element.
- **Found:** If the current element is equal to the desired element, return true or index to the current element.
- **Move:** Otherwise, move to the next element in the collection.
- **Repeat:** Repeat steps 2-4 until we have reached the end of collection.
- **Not found:** If the end of the collection is reached without finding the desired element, return that the desired element is not in the array.

How Does Linear Search Algorithm Work?

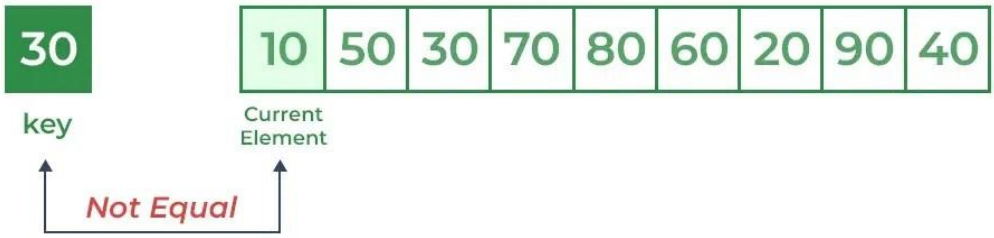
In Linear Search Algorithm,

- Every element is considered as a potential match for the key and checked for the same.
- If any element is found equal to the key, the search is successful and the index of that element is returned.
- If no element is found equal to the key, the search yields "No match found".

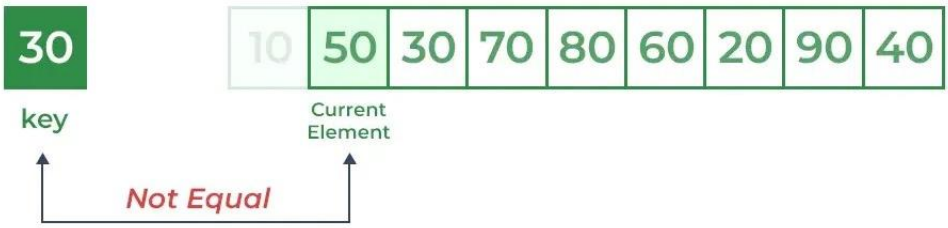
For example: Consider the array **arr[ ] = {10, 50, 30, 70, 80, 20, 90, 40}**
and **key** = 30

Step 1: Start from the first element (index 0) and compare **key** with each element (arr[i]).

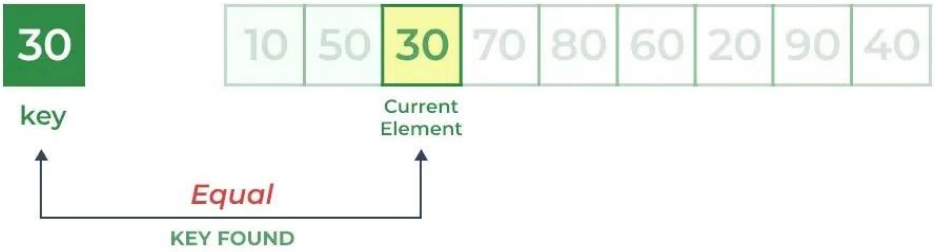
- Comparing key with first element arr[0]. Since not equal, the iterator moves to the next element as a potential match.



- Comparing key with next element arr[1]. SInce not equal, the iterator moves to the next element as a potential match.



Step 2: Now when comparing arr[2] with key, the value matches. So the Linear Search Algorithm will yield a successful message and return the index of the element when key is found (here 2).



<u>Applications of Linear Search:</u>

- **Unsorted Lists:** When we have an unsorted array or list, linear search is most commonly used to find any element in the collection.
- **Small Data Sets:** Linear Search is preferred over binary search when we have small data sets with
- **Searching Linked Lists:** In linked list implementations, linear search is commonly used to find elements within the list. Each node is checked sequentially until the desired element is found.
- **Simple Implementation:** Linear Search is much easier to understand and implement as compared to Binary Search or Ternary Search.

<u>Advantages of Linear Search:</u>

- Linear search can be used irrespective of whether the array is sorted or not. It can be used on arrays of any data type.
- Does not require any additional memory.
- It is a well-suited algorithm for small datasets.

<u>Disadvantages of Linear Search:</u>

- Linear search has a time complexity of O(N), which in turn makes it slow for large datasets.
- Not suitable for large arrays.

What is Binary Search Algorithm?

Binary search is a search algorithm used to find the position of a target value within a **sorted** array. It works by repeatedly dividing the search interval in half until the target value is found or the interval is empty. The search interval is halved by comparing the target element with the middle value of the search space.

Conditions to apply Binary Search Algorithm in a Data Structure:

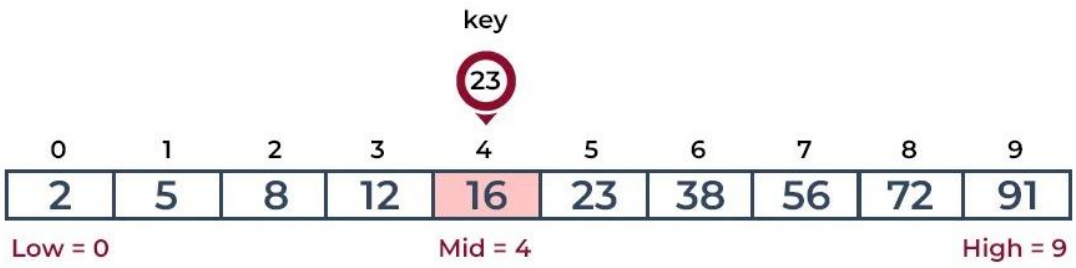
To apply Binary Search algorithm:

- The data structure must be sorted.
- Access to any element of the data structure takes constant time.

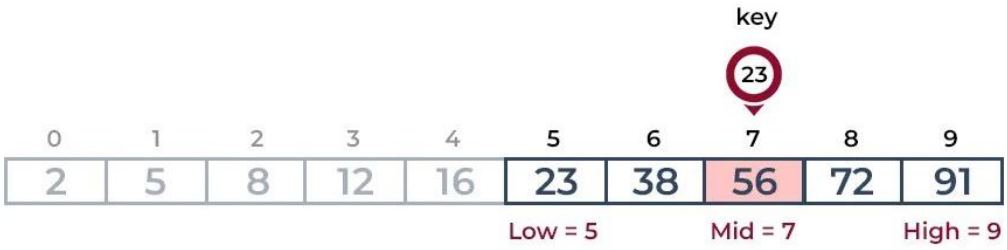
How does Binary Search Algorithm work?

To understand the working of binary search, consider the following illustration:
Consider an array **arr[] = {2, 5, 8, 12, 16, 23, 38, 56, 72, 91}**, and the **target = 23**.
First Step: Calculate the mid and compare the mid element with the key. If the key is less than mid element, move to left and if it is greater than the mid then move search space to the right.

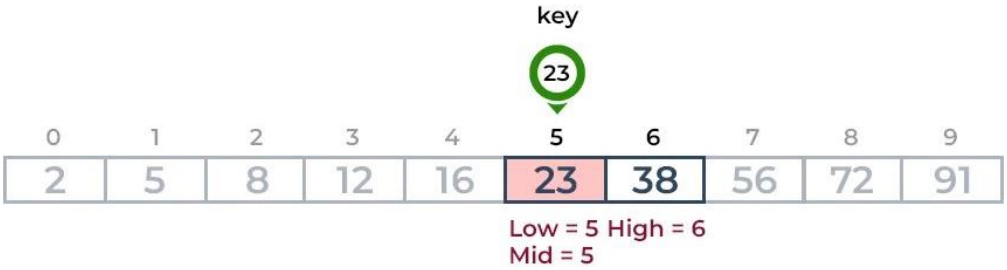
- Key (i.e., 23) is greater than current mid element (i.e., 16). The search space moves to the right.



- Key is less than the current mid 56. The search space moves to the left.



Second Step: If the key matches the value of the mid element, the element is found and stop search.



Applications of Binary Search Algorithm:

- Binary search can be used as a building block for more complex algorithms used in machine learning, such as algorithms for training neural networks or finding the optimal hyperparameters for a model.
- It can be used for searching in computer graphics such as algorithms for ray tracing or texture mapping.
- It can be used for searching a database.

Advantages of Binary Search:

- Binary search is faster than linear search, especially for large arrays.
- More efficient than other searching algorithms with a similar time complexity, such as interpolation search or exponential search.
- Binary search is well-suited for searching large datasets that are stored in external memory, such as on a hard drive or in the cloud.

Disadvantages of Binary Search:

- The array should be sorted.
- Binary search requires that the data structure being searched be stored in contiguous memory locations.
- Binary search requires that the elements of the array be comparable, meaning that they must be able to be ordered.

Sorting

What is Sorting?

Sorting refers to rearrangement of a given array or list of elements according to a comparison operator on the elements. The comparison operator is used to decide the new order of elements in the respective data structure. Sorting means reordering of all the elements either in ascending or in descending order.

Applications of Sorting Algorithms:

- **Searching Algorithms:** Sorting is often a crucial step in search algorithms like binary search, Ternary Search, where the data needs to be sorted before searching for a specific element.
- **Data management:** Sorting data makes it easier to search, retrieve, and analyze.
- **Database optimization:** Sorting data in databases improves query performance.
- **Machine learning:** Sorting is used to prepare data for training machine learning models.
- **Data Analysis:** Sorting helps in identifying patterns, trends, and outliers in datasets. It plays a vital role in statistical analysis, financial modeling, and other data-driven fields.
- **Operating Systems:** Sorting algorithms are used in operating systems for tasks like task scheduling, memory management, and file system organization.

Types of Sorting Algorithm

i) Selection Sort
ii) Bubble Sort
iii) Insertion Sort
iv) Merge Sort
v) Quick Sort

❖ Selection Sort

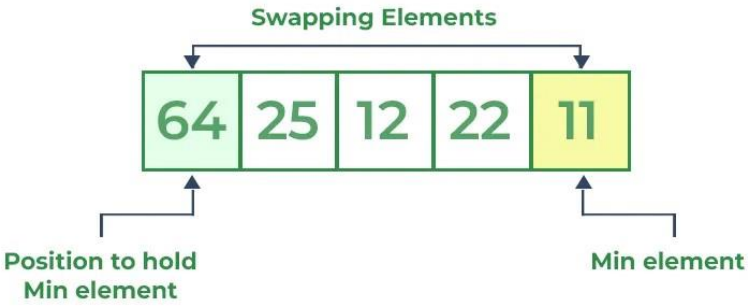
Selection sort is a simple and efficient sorting algorithm that works by repeatedly selecting the smallest (or largest) element from the unsorted portion of the list and moving it to the sorted portion of the list.

How does Selection Sort Algorithm work?

Lets consider the following array as an example: ***arr[ ] = {64, 25, 12, 22, 11}***

First pass:

- *For the first position in the sorted array, the whole array is traversed from index 0 to 4 sequentially. The first position where* ***64*** *is stored presently, after traversing whole array it is clear that* ***11*** *is the lowest value.*
- *Thus, replace 64 with 11. After one iteration* ***11****, which happens to be the least value in the array, tends to appear in the first position of the sorted list.*

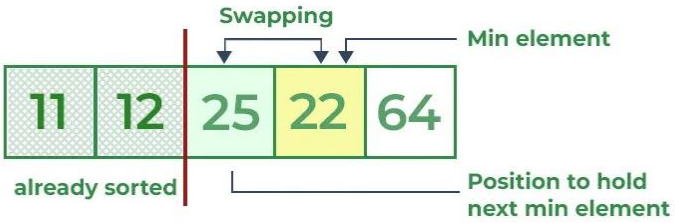


Second Pass:

- For the second position, where 25 is present, again traverse the rest of the array in a sequential manner.
- After traversing, we found that **12** is the second lowest value in the array and it should appear at the second place in the array, thus swap these values.

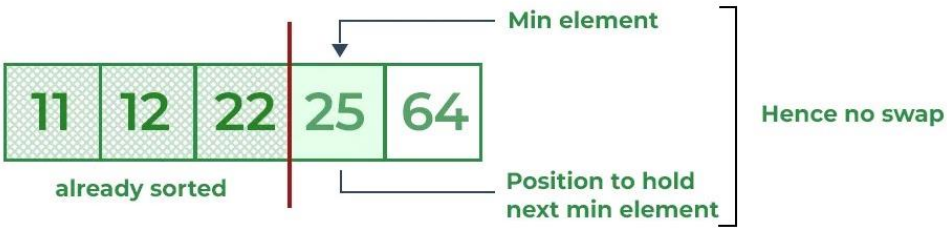
Third Pass:

- Now, for third place, where **25** is present again traverse the rest of the array and find the third least value present in the array.
- While traversing, **22** came out to be the third least value and it should appear at the third place in the array, thus swap **22** with element present at third position.



Fourth pass:

- Similarly, for fourth position traverse the rest of the array and find the fourth least element in the array.
- As **25** is the 4th lowest value hence, it will place at the fourth position.



Fifth Pass:

- At last the largest value present in the array automatically get placed at the last position in the array
- The resulted array is the sorted array.

<u>Advantages of Selection Sort Algorithm</u>

- Simple and easy to understand.
- Works well with small datasets.

<u>Disadvantages of the Selection Sort Algorithm</u>

- Selection sort has a time complexity of O(n^2) in the worst and average case.
- Does not work well on large datasets.
- Does not preserve the relative order of items with equal keys which means it is not stable.

❖ <u>Bubble Sort</u>

Bubble Sort is the simplest sorting algorithm that works by repeatedly swapping the adjacent elements if they are in the wrong order. This algorithm is not suitable for large data sets as its average and worst-case time complexity is quite high.

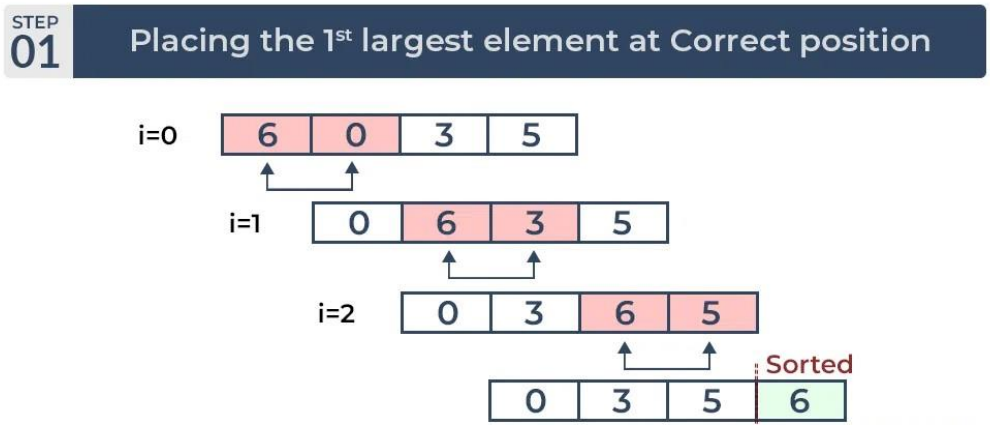
Bubble Sort Algorithm

In Bubble Sort algorithm,

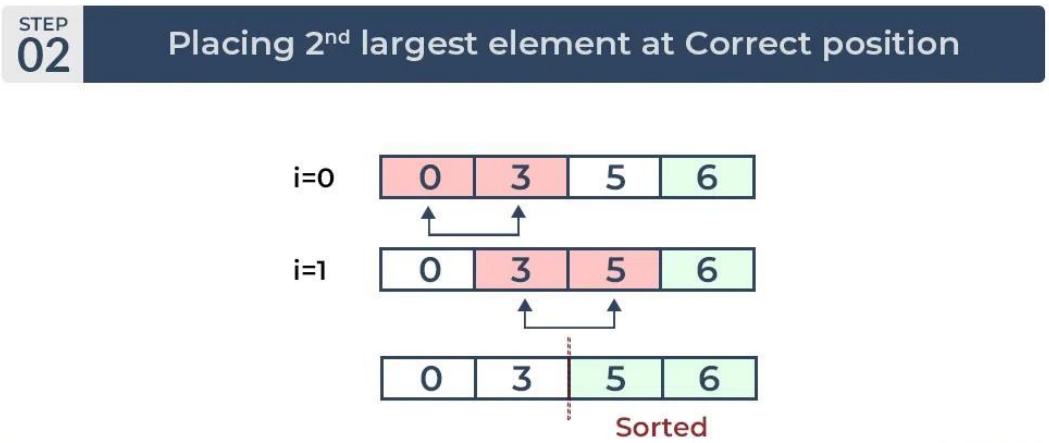
- Traverse from left and compare adjacent elements and the higher one is placed at right side.
- In this way, the largest element is moved to the rightmost end at first.
- This process is then continued to find the second largest and place it and so on until the data is sorted.

How does Bubble Sort Work?
Let us understand the working of bubble sort with the help of the following illustration:
Input: arr[ ] = {6, 0, 3, 5}

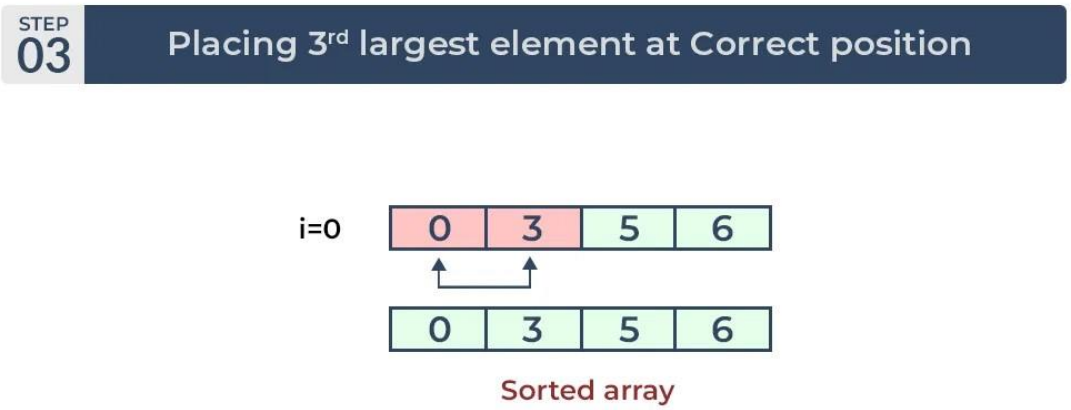
First Pass:
The largest element is placed in its correct position, i.e., the end of the array.



Second Pass:
Place the second largest element at correct position



Third Pass:
Place the remaining two elements at their correct positions.



- **Total no. of passes:** n-1
- **Total no. of comparisons:** n*(n-1)/2

Advantages of Bubble Sort:
- Bubble sort is easy to understand and implement.
- It does not require any additional memory space.
- It is a stable sorting algorithm, meaning that elements with the same key value maintain their relative order in the sorted output.

Disadvantages of Bubble Sort:
- Bubble sort has a time complexity of O(N2) which makes it very slow for large data sets.
- Bubble sort is a comparison-based sorting algorithm, which means that it requires a comparison operator to determine the relative order of elements in the input data set. It can limit the efficiency of the algorithm in certain cases.

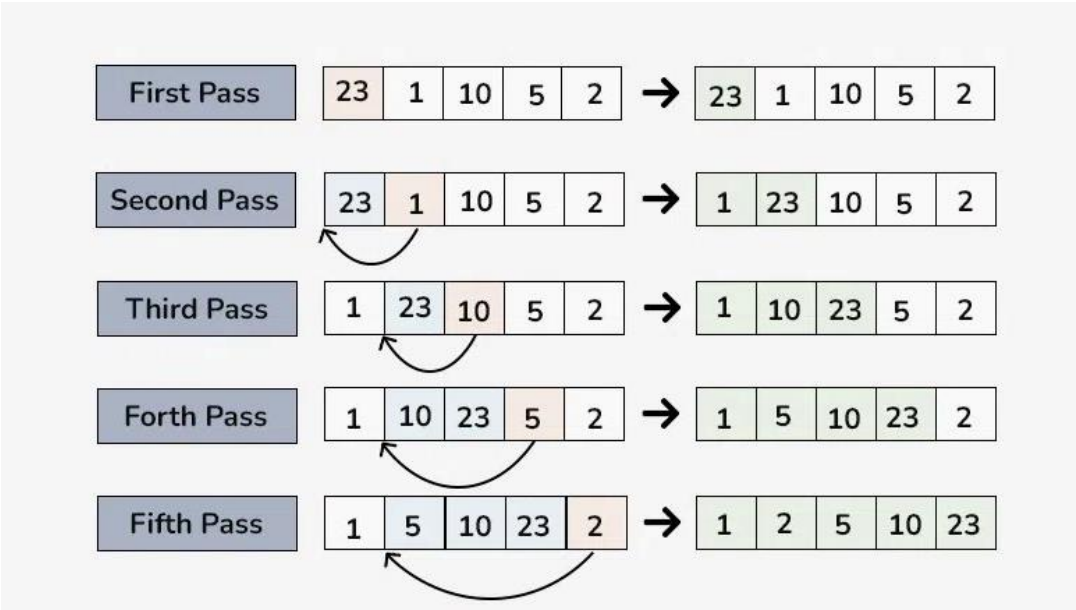
❖ Insertion Sort

Insertion sort is a simple sorting algorithm that works by iteratively inserting each element of an unsorted list into its correct position in a sorted portion of the list. It is a **stable sorting** algorithm, meaning that elements with equal values maintain their relative order in the sorted output.

Insertion Sort Algorithm:
Insertion sort is a simple sorting algorithm that works by building a sorted array one element at a time. It is considered an "**in-place**" sorting algorithm, meaning it doesn't require any additional memory space beyond the original array.
Algorithm:
To achieve insertion sort, follow these steps:
- We have to start with second element of the array as first element in the array is assumed to be sorted.
- Compare second element with the first element and check if the second element is smaller then swap them.
- Move to the third element and compare it with the second element, then the first element and swap as necessary to put it in the correct position among the first three elements.
- Continue this process, comparing each element with the ones before it and swapping as needed to place it in the correct position among the sorted elements.
- Repeat until the entire array is sorted.

Working of Insertion Sort Algorithm:

- Consider an array having elements**: {23, 1, 10, 5, 2}**



First Pass:
- Current element is **23**
- The first element in the array is assumed to be sorted.
- The sorted part until **0th** index is : **[23]**

Second Pass:
- Compare **1** with **23** (current element with the sorted part).
- Since **1** is smaller, insert **1** before **23**.
- The sorted part until **1st** index is: **[1, 23]**

Third Pass:
- Compare **10** with **1** and **23** (current element with the sorted part).
- Since **10** is greater than **1** and smaller than **23**, insert **10** between **1** and **23**.
- The sorted part until **2nd** index is: **[1, 10, 23]**

Fourth Pass:
- Compare **5** with **1**, **10**, and **23** (current element with the sorted part).
- Since **5** is greater than **1** and smaller than **10**, insert **5** between **1** and **10**.
- The sorted part until **3rd** index is**: [1, 5, 10, 23]**

Fifth Pass:
- Compare **2** with **1, 5, 10**, and **23** (current element with the sorted part).
- Since **2** is greater than **1** and smaller than **5** insert **2** between **1** and **5**.
- The sorted part until **4th** index is: **[1, 2, 5, 10, 23]**

Final Array:
- The sorted array is: **[1, 2, 5, 10, 23]**

Advantages of Insertion Sort:
- Simple and easy to implement.
- Stable sorting algorithm.
- Efficient for small lists and nearly sorted lists.
- Space-efficient.

Disadvantages of Insertion Sort:
- Inefficient for large lists.
- Not as efficient as other sorting algorithms (e.g., merge sort, quick sort) for most cases.

Applications of Insertion Sort:
Insertion sort is commonly used in situations where:
- The list is small or nearly sorted.
- Simplicity and stability are important.

Comparison Between Searching and Sorting Algorithm:

S.No.	Searching Algorithm	Sorting Algorithm
1.	Searching Algorithms are designed to retrieve an element from any data structure where it is used.	A Sorting Algorithm is used to arranging the data of list or array into some specific order.
2.	These algorithms are generally classified into two categories i.e. Sequential Search and Interval Search.	There are two different categories in sorting. These are Internal and External Sorting.
3.	The worst-case time complexity of searching algorithm is O(N).	The worst-case time complexity of many sorting algorithms like Bubble Sort, Insertion Sort, Selection Sort, and Quick Sort is $O(N^2)$.
4.	There is no stable and unstable searching algorithms.	Bubble Sort, Insertion Sort, Merge Sort etc are the stable sorting algorithms whereas Quick Sort, Heap Sort etc are the unstable sorting algorithms.
5.	The Linear Search and the Binary Search are the examples of Searching Algorithms.	The Bubble Sort, Insertion Sort, Selection Sort, Merge Sort, Quick Sort etc are the examples of Sorting Algorithms.

****